Laboratorul 3 - MPI - Calculul sumei a două numere mari

Introducere

Scop

- Dezvoltarea unui program bazat pe MPI pentru calculul sumei a două numere mari.

Restricții

- Fără deadlock!
- Implementare utilizând C++11.

Structura fișierelor

- Două fișiere de intrare care conțin numerele:
  - Numerele sunt reprezentate ca un tablou de cifre (valori întregi fără semn) în care cifra cea mai puțin semnificativă este pe poziția 0.
  - Numerele sunt stocate în ordine inversă (cifra cea mai semnificativă se află la poziția 0).
  - Dimensiunile numerelor sunt egale.
- Fișierul de ieșire va conține rezultatul sumei celor două numere.

Abordări Implementate

Varianta 0 (Secvențială)

- Programul calculează suma a două numere mari parcurgând fiecare cifră și calculând transportul.
- Dacă transportul este 1, acesta va fi adăugat la următoarea cifră.

Varianta 1 (Distribuție pe procese)

- **Pași principali**:
  1. Procesul curent inițial: `id_proces_curent=1`.
  2. Procesul părinte:
     - Citește cifrele până când toate au fost procesate (cifrele sunt împărțite la `(N/(p-1))` pe fiecare proces).
     - Trimite datele la `id_proces_curent`, apoi incrementează `id_proces_curent`.
  3. Procesul copil:
     - Primește cifrele de la părinte și calculează suma, determinând și transportul.
     - În afară de ultimul proces, fiecare proces trimite transportul următorului.
     - Fiecare proces trimite rezultatul său părintelui (ultimul proces trimite și transportul final).
- **Avantaj**: Uneori, transportul primit nu afectează cifra curentă, astfel că nu este necesar să se mai efectueze operații suplimentare. Aceasta minimizează numărul de operații de adunare.

Varianta 2 (Broadcast și Gather)

- **Pași principali**:
    1. Procesul părinte:
        - Citește numerele și le stochează în două tablouri de cifre.
        - Dacă unul dintre numere are mai puține cifre, adaugă padding (zero-uri la început).
        - Dacă `N=max{N_1,N_2}` nu se împarte exact la `(p-1)`, se adaugă padding astfel încât numerele să fie divizibile la `(p-1)`.
        - Trimite numărul de cifre prin broadcast.
    2. Procesul copil:
        - Calculează suma cifrelor și propagă transportul la procesul următor (în afară de ultimul).
        - Ultimul proces trimite transportul final la părinte, iar datele sunt returnate părintelui prin `MPI_Gather`.

Varianta 3 (Asincron)

- **Procesul părinte**:
    - Citește numerele și creează cereri asincrone pentru a coordona trimiterea datelor.
    - Trimite fiecare „chunk” de date către procese, împărțind atât numărul mai mare, cât și cel mai mic.
    - Dacă ajunge la o poziție în numărul mai mic unde acesta nu mai continuă, oprește trimiterea.
    - Așteaptă rezultatele și transportul de la procese, apoi scrie rezultatul final în fișier.
- **Procesul copil**:
    - Primește „chunk”-urile de cifre și calculează transportul.
    - Dacă transportul este necesar pentru următorul proces, acesta este trimis.
    - La final, fiecare proces trimite rezultatul înapoi la procesul părinte; ultimul proces trimite și transportul final.

Complexitate de spațiu

- O(max(N1, N2)) pentru fiecare variantă.

Tehnologii folosite

- C++17
- MPI (Message Passing Interface)

Teste efectuate

Am rulat fiecare variantă pentru următoarele teste:

1. **Numere egale**: Numar1 = "123456789123456789", Numar2 = "123456789123456789".
2. **Numere aleatorii, dimensiuni egale**: N_1=1000 și N_2=1000 (cifre aleatorii).
3. **Dimensiuni diferite**: N_1=100 și N_2=100000 (cifre aleatorii).
4. **Numere speciale**: N_1=N_2=16, Numar1="9999 4444 4444 9999", Numar2="9999 5555 5555 9999", cu 4 procese.

Număr de procese: 4, 8, 16

Clase

Iată o descriere scurtă a claselor și funcțiilor prezentate:

1. **Clasele de Varianta**

`SequentialVariant0`

- **Rol**: Implementarea secvențială a sumei a două numere mari.
- **Membri**:
  - `path_first_number`, `path_second_number`, `resultFile`: calea fișierelor de intrare și de ieșire.
- **Funcții**:
  - `determineSum()`: Calculează suma celor două numere mari citite din fișiere și scrie rezultatul în fișierul de ieșire.

`Variant1`

- **Rol**: Realizează suma în mod paralel, folosind procese MPI pentru a aduna cifrele și a propaga transportul.
- **Membri**:
  - `argc`, `argv`: Parametrii pentru inițializarea MPI.
  - `path_first_number`, `path_second_number`, `resultFile`: Calea fișierelor.
- **Funcții**:
  - `determineSum()`: Inițializează MPI, distribuie cifrele către procese și colectează rezultatul.
  - `sumaADouaNumere()`: Adună două numere mari stocate în vectori.
  - `adaugaCarry()`: Adaugă transportul în cifrele unui număr, pornind de la poziția dorită.

`Variant2Scatter`

- **Rol**: Distribuie bucăți (chunks) de date către procesele MPI prin `MPI_Scatter` și le adună cu propagarea transportului.
- **Membri**:
  - `argc`, `argv`: Parametrii pentru MPI.
  - `path_first_number`, `path_second_number`, `resultFile`: Calea fișierelor.
- **Funcții**:
  - `determineSum()`: Distribuie bucăți ale numerelor mari, folosește scatter pentru distribuirea cifrelor, calculează rezultatul și reunește rezultatele folosind `MPI_Gather`.
  - `sumaADouaNumere()`: Adună două numere stocate în vectori și calculează transportul.
  - `adaugaCarry()`: Gestionează transportul la nivelul cifrelor într-un număr.

`Variant3Async`

- **Rol**: Versiunea asincronă a sumei în paralel, folosind cereri asincrone pentru a optimiza comunicarea și pentru a gestiona transportul între procese.
- **Membri**:
  - `argc`, `argv`: Parametrii pentru MPI.
  - `path_first_number`, `path_second_number`, `resultFile`: Calea fișierelor.
- **Funcții**:
  - `determineSum()`: Distribuie bucăți de date către procesele MPI, utilizează comunicația asincronă pentru a colecta rezultatul și a gestiona transportul între procese.

- `sumaADouaNumere()`: Adună două numere, folosind cifrele din două vectori și calculează transportul.
- `adaugaCarry()`: Adaugă transportul într-un număr stocat într-un vector de cifre, pornind de la poziția dorită.

2. **Clasa `Utils`**

Rol:

Utilitar pentru funcții de citire și scriere în fișiere.

Funcții:

- `read_chunk()`, `read_chunk_v()`: Citește un bloc (chunk) de cifre din fișier și le returnează într-un vector.
- `read_file()`: Citește un număr întreg din fișier și îl returnează ca un vector de cifre.
- `read_number_of_digits()`: Returnează numărul de cifre dintr-un fișier.
- `write_result()`: Scrie rezultatul adunării în fișierul de ieșire.

3. **Funcții Independente**

`areFilesEqual()`

- **Rol**: Verifică dacă două fișiere sunt identice comparându-le byte cu byte.

Analiza rezultatelor:

Variant	NumberOfThreads	Time	TestCase
Variant0	1	0.00759193	T1
Variant1	4	0.00923806	T1
Variant1	8	0.01074311	T1
Variant1	16	0.01212781	T1
Variant2	4	0.01012925	T1
Variant2	8	0.00995305	T1
Variant2	16	0.01172159	T1
Variant3	4	0.00650717	T1
Variant3	8	0.01040181	T1
Variant3	16	0.01195027	T1
Variant0	1	0.00574536	T2
Variant1	4	0.00917119	T2
Variant1	8	0.0106271	T2

Variant	NumberOfThreads	Time	TestCase
Variant1	16	0.0115485	T2
Variant2	4	0.00953167	T2
Variant2	8	0.01032437	T2
Variant2	16	0.01164709	T2
Variant3	4	0.00927951	T2
Variant3	8	0.00906072	T2
Variant3	16	0.01066942	T2
Variant0	1	0.0067324	T3
Variant1	4	0.136802	T3
Variant1	8	0.1395339	T3
Variant1	16	0.1460864	T3
Variant2	4	0.1439644	T3
Variant2	8	0.1455699	T3
Variant2	16	0.1491246	T3
Variant3	4	0.1462325	T3
Variant3	8	0.1485217	T3
Variant3	16	0.1521785	T3
Variant0	1	0.0069333	T4
Variant1	4	0.01076145	T4
Variant1	8	0.01060063	T4
Variant1	16	0.01231364	T4
Variant2	4	0.00942044	T4
Variant2	8	0.0096953	T4
Variant2	16	0.01170925	T4
Variant3	4	0.00939875	T4
Variant3	8	0.00903936	T4
Variant3	16	0.01151501	T4

Iată analiza datelor și principalele concluzii extrase:

1. **Execuția Secvențială (Varianta0)**

- **Stabilitatea Performanței:** `Varianta0` (execuție secvențială) arată timpi de execuție constanți pe toate testele, așa cum era de așteptat, întrucât nu se folosește procesarea paralelă.
- **Intervalul de Timp de Execuție:** Timpii de execuție sunt, în general, mai mici comparativ cu versiunile paralele, deoarece nu există un cost suplimentar de comunicare MPI. Totuși, `Varianta0` nu se scalează odată cu numărul de procese, ceea ce o face nepotrivită pentru intrări mari ce necesită procesare rapidă.

2. **Varianta1 (Adunare Simplă Paralelă cu Propagarea Transportului)**

- **Scalabilitate cu Numărul de Procese:** `Varianta1` arată îmbunătățiri ușoare ale timpului de execuție atunci când trece de la 4 la 16 procese, însă câștigurile scad peste 8 procese. Motivul poate fi faptul că costul propagării transportului între procese depășește avantajele vitezei adăugării unor noi procese.
- **Impactul Intrărilor Mari (T3):** Timpii de execuție pentru `T3` (caz de intrare mare) sunt semnificativ mai mari pe toate numerele de procese (de exemplu, 4 procese: 0.136802, 16 procese: 0.1460864), ceea ce indică faptul că varianta întâmpină blocaje la intrările mari din cauza costului propagării transportului între procese.

3. **Varianta2 (Distribuție a Chunks cu Umplere Dinamică)**

- **Eficiență Paralelă:** `Varianta2` obține, în general, timpi de execuție mai mici comparativ cu `Varianta1` pentru aceleași numere de procese, ceea ce indică faptul că abordarea sa de distribuție a chunk-urilor și umplere este mai eficientă.
- **Impactul Tipului de Test:** Performanța rămâne relativ constantă pentru `T1`, `T2` și `T4`. Totuși, `T3` arată o creștere mare a timpului de execuție (de exemplu, 4 procese: 0.1439644, 16 procese: 0.1491246), ceea ce indică faptul că `Varianta2` întâmpină dificultăți la intrările mari, probabil din cauza complexității comunicării și umplerii.
- **Performanță Optimă la Număr Mediu de Procese:** Pentru cazurile de test mai mici, timpii de execuție se îmbunătățesc de la 4 la 8 procese, dar arată beneficii mai mici la 16 procese, probabil din cauza costului suplimentar de comunicare.

4. **Varianta3 (Comunicație Asincronă și Gestionarea Transportului)**

- **Cea Mai Bună Scalabilitate pe Procese:** `Varianta3` prezintă cele mai mari îmbunătățiri ale timpilor de execuție pe măsură ce crește numărul de procese, în special la `T1` și `T2`. De exemplu, la `T1`, timpul de execuție scade de la 0.01040181 (8 procese) la 0.00650717 (4 procese).
- **Gestionarea Intrărilor Mari (T3):** Deși `Varianta3` are totuși timpi mai mari pentru `T3`, acești timpi sunt puțin mai mici comparativ cu `Varianta1` și `Varianta2` pentru intrările mari, indicând că strategia sa de comunicare asincronă reduce unele blocaje.
- **Consistență la Testul T4:** Timpii de execuție pentru `T4` rămân relativ stabili pe diferite numere de procese, arătând că `Varianta3` poate gestiona eficient intrările de date echilibrate.

5. **Concluzii Generale și Recomandări**

- **Optimă pentru Intrări Mici:** Pentru cazurile de intrare mici sau medii, `Varianta0` sau `Varianta3` cu un număr mai mic de procese (4 sau 8) sunt opțiuni optime. `Varianta0` performează bine datorită simplității, iar `Varianta3` utilizează eficient comunicarea asincronă.
- **Optimă pentru Intrări Mari:** `Varianta3` este cea mai scalabilă alegere pentru intrări mari, deoarece gestionează propagarea transportului mai eficient prin comunicare asincronă.

- **Beneficii Reduse la Număr Mare de Procese:** Toate variantele prezintă beneficii reduse ale timpilor de execuție atunci când se trece de la 8 la 16 procese, sugerând că, pentru cele mai multe cazuri, 4-8 procese sunt optimul pentru câștiguri de performanță fără un cost excesiv de comunicație MPI.

În concluzie, `Varianta3` **cu 4-8 procese** este cea mai echilibrată abordare pentru a gestiona eficient atât intrările mici, cât și cele mari.